

Dynamic Collaborative Scheduling System for New Energy Logistics Fleets Based on Graph Algorithms and Reinforcement Learning

Zepeng Lin
AI Class2
202430690300

Guanyu Xin
AI Class2
202464870413

Yuxuan Xue
AI Class2
202430841574

<https://github.com/oplisty/Data-Structure-HW>

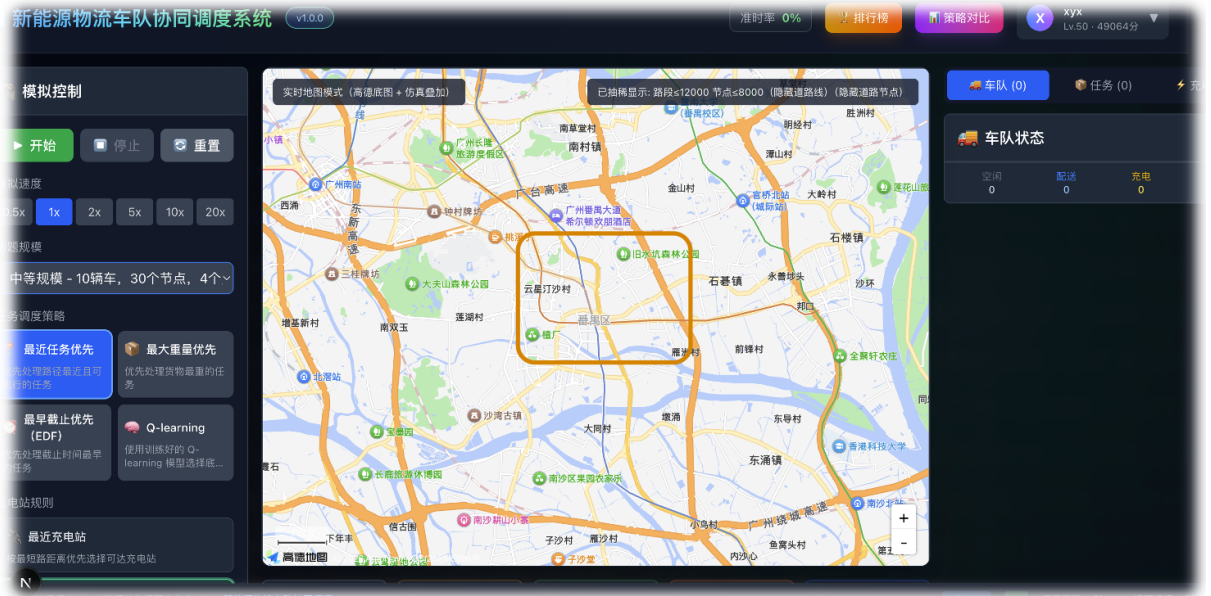


Figure 1. Visualization interface of the dynamic collaborative scheduling system for new energy logistics fleets.

Abstract

With the rapid growth of urban instant delivery, scheduling for new energy logistics fleets faces increasing challenges from dynamic task arrivals, limited vehicle resources, and complex charging constraints. This paper presents a dynamic collaborative scheduling system based on graph modeling and intelligent decision methods. The urban road network is represented as a weighted graph, and a discrete-time simulation framework is built by considering battery limits, load capacity, task deadlines, and charging station congestion. For path planning and reachability analysis, the system applies Dijkstra, A*, and

RRT, while at the scheduling level it implements several heuristic strategies, including nearest-task-first, maximum-weight-first, earliest-deadline-first and Q-learning-based hyper-heuristic. In addition, an offline MILP module under full-information settings is developed for small-scale cases to generate near-optimal solutions and provide a baseline for comparison with online scheduling strategies. The system follows a “simulation engine + backend + visualization frontend” architecture. Experimental show that it can effectively capture the key constraints and operating process of new energy logistics delivery, and that different strategies exhibit clear differences in task completion rate, total reward, path length, and charging load. Overall,

this work develops a research-relevant platform for dynamic scheduling of new energy logistics fleets, providing a practical basis for future studies on multi-agent coordination, exact optimization, and more complex urban delivery scenarios.

1. Introduction

With the rapid development of e-commerce, on-demand local delivery, and smart logistics, urban delivery systems are facing stricter timeliness requirements and more complex operational constraints. Compared with traditional fuel vehicles, new energy logistics vehicles offer advantages such as lower emissions, higher energy efficiency, and policy support, and are becoming an important part of urban delivery. However, they also introduce new challenges, including limited battery capacity, time-consuming charging, and limited charging-station resources with possible congestion, which make vehicle routing and task scheduling more difficult.

In real logistics scenarios, delivery tasks are usually not fully known in advance but arrive dynamically over time. Under constraints such as limited fleet size, vehicle load limits, continuous battery consumption, task deadlines, and charging-station queues, managers must decide in real time which vehicle should serve which task, how to plan routes, when to charge, and whether multiple vehicles should collaborate on the same task. Therefore, this is not simply a shortest-path problem, but a comprehensive problem involving graph search, resource-constrained scheduling, dynamic decision-making, and system simulation. Based on this background, we aim to design and implement a complete system platform to simulate and analyze the performance of different scheduling strategies in new energy logistics scenarios.

1.1. Dynamic Modeling of Vehicles and Tasks

In this project, vehicles, tasks, depots, and charging stations are modeled in a unified way at the entity level. Each vehicle includes attributes such as current location, battery level, battery capacity, maximum load capacity, current load, speed, and energy consumption per unit distance. Each task includes information such as release time, deadline, task location, cargo weight, and status. During the simulation, tasks are released dynamically as time progresses, rather than being provided to the scheduler all at once, so that the system can better reflect the online decision-making process in real delivery scenarios.

1.2. Graph-Based Road Network and Path Planning

This project uses graphs as the core data structure and models the road network as a weighted graph, where nodes represent road intersections, depots, task points, and charging stations, and edges represent road connections together with their distance or travel cost. For path planning, the system implements Dijkstra’s algorithm as a stable and reliable shortest-path baseline, while A* is introduced to improve goal-directed search efficiency. RRT is also included as an extended experimental algorithm to demonstrate the scalability of the system in path planning. With these algorithms, the system can support shortest-path queries, reachability checking, return-to-depot feasibility checking, and nearest charging station search.

1.3. Reward and Charging Constraints

This project implements a task reward calculation and overdue penalty mechanism in the simulation environment, and models each charging station as a resource node with a fixed number of chargers, occupancy status, and a waiting queue. The system not only checks whether a vehicle can complete a task and return to the depot, but also automatically triggers charging-station selection when the battery level is low. In station selection, it considers not only distance, but also queue length and station occupancy, so as to better reflect the practical complexity of new energy logistics delivery.

1.4. Scheduling, Experiments, and Visualization

This project implements several heuristic scheduling strategies, including nearest-task-first, maximum-weight-first, and earliest-deadline-first, and supports different scenario settings such as small-scale, medium-scale, and large-scale cases. In addition, the project further explores the use of Q-learning for dynamic scheduling decisions and builds an offline MILP module to solve global plans for small-scale scenarios. Based on these components, the project also provides real-time visualization of the map, tasks, vehicles, charging stations, and statistical indicators through a web frontend, so that the system can be used not only for simulation but also for demonstration and result analysis.

1.5. Summary

This project adopts an overall architecture of “simulation engine + algorithm modules + real-time interface + web-based visualization frontend.” At the simulation level, the system maintains the graph-based road network, vehicle states, task states, charging station states, and the logic of time advancement. At each time step, it sequentially performs task release, task expiration checking, scheduling decisions, vehicle

movement, battery consumption, charging queue management, and charging completion, forming a complete discrete-time simulation loop. At the algorithm level, path planning and task scheduling are designed as separate modules: path planning handles shortest-path search, target reachability, and energy feasibility analysis, while scheduling strategies determine dispatch actions based on visible tasks, vehicle states, and system constraints. This modular design allows different algorithms to be compared fairly in the same environment and makes future extensions easier.

To address the course requirement for advanced methods, the project further implements two extended approaches. One is a Q-learning-based hyper-heuristic strategy selection method, which learns which low-level heuristic rule should be used in different situations through discrete state encoding. The other is an MILP-based offline solver, which computes globally near-optimal routes and scheduling plans for small-scale cases under full-information settings, and compares them with the performance of online strategies. In addition, the system provides real-time visualization through frontend-backend interaction: the backend updates the environment and records logs, while the frontend displays map nodes, vehicle movements, task-state changes, charging station queues, and statistical indicators. Users can also switch scheduling strategies, replay offline solutions, and perform Q-learning training and inference through the interface, which improves both interactivity and presentation quality.

2. Framework

This paper designs a layered system architecture for the dynamic collaborative scheduling of new energy logistics fleets. Overall, the system can be divided into five core layers from bottom to top: the data resource layer, the algorithm layer, the simulation engine layer, the backend service layer, and the frontend visualization layer. Although each layer is relatively independent, they are connected through unified data interfaces and state transition logic to form a complete closed loop. This layered design has two main advantages. First, it decouples functions such as road modeling, path planning, scheduling decision-making, simulation execution, and interface presentation, thereby reducing overall system complexity. Second, it allows different path planning algorithms, scheduling strategies, and optimization methods to be flexibly replaced and compared within the same framework, which not only meets the implementation requirements of a course project but also supports further research and future extensions.

2.1. Data Layer

The data resource layer provides a unified representation of the environment for the whole system and serves as the foundation for subsequent path planning, task scheduling, and state simulation. To satisfy the course requirement of using a graph structure for road representation and routing, this paper first models the urban delivery scenario as a weighted directed graph

$$G = (V, E, W),$$

where the node set V represents road intersections, depots, candidate task locations, and charging stations; the edge set $E \subseteq V \times V$ represents road connectivity; and the weight function $W : E \rightarrow \mathbb{R}^+$ describes the distance or travel cost of each edge. Based on this representation, the entire logistics environment can be mapped into a discrete graph space, which provides a computable foundation for shortest-path search, reachability analysis, and energy consumption evaluation.

In the implementation, the system denotes the depot as node $v_d \in V$, the set of charging stations as

$$S = \{s_1, s_2, \dots, s_m\} \subseteq V,$$

and generates dynamic tasks from the candidate task node set $V_T \subseteq V$. For any task τ_i , it can be represented as the following four-tuple:

$$\tau_i = (r_i, v_i, w_i, d_i),$$

where r_i is the release time of the task, $v_i \in V_T$ is the task location, w_i is the cargo weight, and d_i is the deadline. This representation allows each task to include not only spatial information but also explicit time constraints, thereby supporting online scheduling decisions in dynamic arrival scenarios.

Furthermore, the data resource layer maintains the static parameter sets of vehicles and stations. For example, for vehicle k , its state parameters can be defined as

$$\mathbf{x}_k = (b_k^{\max}, b_k, c_k, v_k^{\text{cur}}, \eta_k),$$

where $b_k^{\max}$ and b_k denote the maximum battery level and current battery level, respectively; c_k is the load capacity limit; v_k^{cur} is the current node location; and η_k is the energy consumption per unit distance. For charging station s_j , the system records attributes such as the number of chargers, queue length, and charging rate. In this way, the data resource layer provides not only the static topology of the road network but also a unified description of the parameters of tasks, vehicles, and charging facilities.

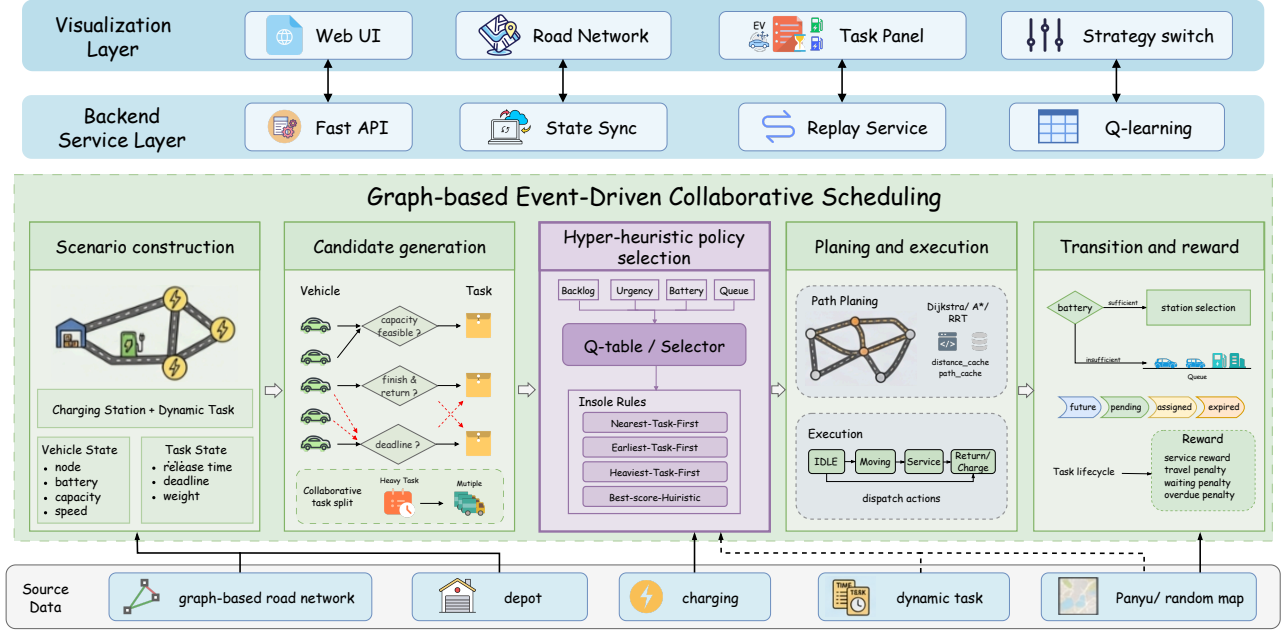


Figure 2. Overall framework of the dynamic collaborative scheduling system for new energy logistics fleets. The figure illustrates the relationships among the visualization frontend, backend service layer, simulation engine, scheduling strategies, path planning algorithms, and data resources.

2.2. Algorithm Layer

After the data resource layer completes the formal representation of the environment, the algorithm layer is further responsible for generating executable routing and scheduling decisions on the graph-based road network $G = (V, E, W)$. In this paper, this layer is divided into two closely coupled basic modules: the path planning module and the task scheduling module.

2.2.1. Path Planning

For any vehicle k and target node $v \in V$, the goal of the path planning module is to find a feasible path $P_k(v_k^{\text{cur}}, v)$ on graph G from the current location v_k^{cur} to the target node v . Let the total cost of a path P be

$$C(P) = \sum_{e \in P} W(e),$$

then the shortest path problem can be written as

$$P_k^*(v_k^{\text{cur}}, v) = \arg \min_{P \in \mathcal{P}(v_k^{\text{cur}}, v)} C(P),$$

where $\mathcal{P}(v_k^{\text{cur}}, v)$ denotes the set of all feasible paths from v_k^{cur} to v .

Based on this unified objective, the system implements three path planning baselines.

- Dijkstra: A classic shortest-path algorithm for weighted graphs. It directly minimizes the total

path cost $C(P)$ and can stably return the global optimal path. Therefore, it is used as the standard shortest-path baseline in the system.

- A*: This method introduces a heuristic evaluation function $h(u, v)$ on top of Dijkstra and uses

$$f(n) = g(n) + h(n)$$

as the search priority, where $g(n)$ is the known accumulated cost from the start node to the current node n , and $h(n)$ is the heuristic estimated cost from n to the target node. This method maintains good search quality while reducing unnecessary node expansions, thereby improving the efficiency of each query.

- RRT: A sampling-based search method. It does not directly guarantee a strict shortest path in the graph sense, but explores a large state space through random tree expansion. This provides an interface for future research under more complex path planning settings.

In the new energy delivery scenario, path planning is not only used for travel from the current node to a task node, but also needs to support feasibility checking for returning to the depot after task completion and visiting a charging station when the battery is low. Therefore, for a task $\tau_i = (r_i, v_i, w_i, d_i)$, the system not only computes $C(P_k^*(v_k^{\text{cur}}, v_i))$, but also further evaluates

whether the vehicle satisfies the basic energy consumption constraint. Let η_k denote the energy consumption per unit distance of vehicle k . Then the energy consumption along path P can be approximately written as

$$E_k(P) = \eta_k C(P).$$

Only when the current battery level b_k is sufficient to support the planned trip is the path regarded as executable.

2.2.2. Task Scheduling

After feasible paths are obtained, the scheduling module further determines which task should be assigned to which vehicle. For the currently visible task set $\mathcal{T}_t$ and the current state $\mathbf{x}_k$ of vehicle k , a baseline scheduling strategy essentially defines a priority score function $\phi_k(\tau_i)$ over all candidate tasks and selects

$$\tau_k^* = \arg \max_{\tau_i \in \mathcal{T}_t} \phi_k(\tau_i)$$

as the task with the highest priority at the current time. The difference between different baselines lies in how the score function $\phi_k(\tau_i)$ is defined.

- Nearest-task-first strategy: This strategy uses the minimum path cost as the main criterion. Its policy can be written as

$$\tau_k^* = \arg \min_{\tau_i \in \mathcal{T}_t} C(P_k^*(v_k^{\text{cur}}, v_i)).$$

- Maximum-weight-first strategy: This strategy focuses on giving priority to high-load tasks. For a task $\tau_i = (r_i, v_i, w_i, d_i)$, its policy can be written as

$$\tau_k^* = \arg \max_{\tau_i \in \mathcal{T}_t, w_i \leq c_k} w_i.$$

- Earliest-deadline-first strategy: This strategy takes task urgency as the main consideration, and can be expressed as

$$\tau_k^* = \arg \min_{\tau_i \in \mathcal{T}_t} d_i.$$

Beyond the above baselines, we further implement an online reinforcement learning policy selection module based on Q-learning, as well as an offline global optimization module based on MILP under full information. These two modules correspond to two different methodological paradigms: online decision-making based on interactive learning and offline solving based on exact optimization, as summarized in Algorithm 1 and Algorithm 2.

2.3. Simulation Layer

The simulation engine layer is responsible for transforming the static environment defined in the data resource layer and the scheduling actions produced by the

algorithm layer into an executable time-based process. It is the core component that enables dynamic closed-loop simulation in the whole system. Unlike methods that focus only on a single path query or a local dispatch decision, this layer explicitly models state variables that evolve over time, including task release, vehicle movement, battery consumption, charging queues, and task completion. In this way, the new energy logistics scheduling problem can be continuously solved and evaluated along a unified timeline.

In implementation, the system adopts a discrete-time update mechanism. Let the global environment state at time t be denoted by $\mathcal{S}_t$, and let the joint action generated by the algorithm layer be denoted by $\mathcal{A}_t$. Then, the simulation engine can be formalized as a state transition process:

$$\mathcal{S}_{t+1} = \Phi(\mathcal{S}_t, \mathcal{A}_t),$$

where $\Phi(\cdot)$ denotes the environment transition operator jointly defined by task updates, vehicle movement, energy consumption updates, charging scheduling, and statistical recording. More specifically, at each time step, the system first adds newly released tasks into the pending-task set according to their release times and removes tasks that have already passed their deadlines. It then calls the scheduler to generate dispatch actions based on the current vehicle states and the candidate task set. After that, the system executes path movement and task service, while updating vehicle positions, remaining battery levels, load states, and task completion status. Finally, it processes charging-station events such as queuing, charger occupation, and battery replenishment, and synchronously refreshes system-level statistics.

The key role of this layer is to extend path planning and task scheduling from a static selection problem to a dynamic evolution problem under resource constraints. For example, when vehicle k travels along path P , its battery level decreases continuously according to the energy consumption model defined earlier, namely $E_k(P) = \eta_k C(P)$. When b_k is no longer sufficient to support the planned trip, the simulation engine triggers charging-station selection and queueing logic, and updates the waiting state under limited station capacity. Similarly, for dynamically released tasks, the simulation engine determines not only when a task enters the assignable set, but also whether it becomes a penalty item because of deadline violation. As a result, metrics such as task reward, vehicle utilization, path length, and charging load can all be computed consistently within a unified simulation framework.

Algorithm 1: Q-learning-based Policy Selection

Require: training episodes M , learning rate α , discount factor γ , initial exploration rate ϵ , decay factor ρ , discrete state space S , action space A

Ensure: learned Q-table Q

- 1: Initialize $Q(s, a) = 0$, for all $(s, a) \in S \times A$
- 2: for episode = 1 to M do
- 3: Reset the initial state s_0
- 4: while the episode is not terminated do
- 5: Generate a random number $k \in [0, 1]$
- 6: choose a random action a_t is random if $k < \epsilon$ else $a_t = \arg \max_a Q(s_t, a)$
- 7: Execute action a_t
- 8: Observe reward r_t , next state s_{t+1} , and the termination flag
- 9: Update the Q-table:
- 10: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$
- 11: $s_t \leftarrow s_{t+1}$
- 12: end while
- 13: $\epsilon \leftarrow \max(\epsilon_{\min}, \rho \cdot \epsilon)$
- 14: end for
- 15: return Q

Algorithm 2: Offline MILP-based Global Planning Given the full-information task set $\mathcal{T}$, vehicle set $\mathcal{V}$, expanded node set $\mathcal{N}$, and arc cost matrix C , the offline planner solves the following mixed-integer linear program:

$$\min \sum_{v \in \mathcal{V}} \sum_{(i,j) \in \mathcal{A}} C_{ij} x_{vij} + \lambda_1 \sum_{v \in \mathcal{V}} \sum_{n \in \mathcal{T}} \text{tardy}_{vn} + \lambda_2 T_{\max}. \quad (1)$$

The optimization is subject to

$$\sum_{v \in \mathcal{V}} \text{visit}_{vn} = 1, \quad \forall n \in \mathcal{T}, \quad (2)$$

$$\sum_{j: (s,j) \in \mathcal{A}} x_{vsj} = \text{use}_v, \quad \sum_{i: (i,t) \in \mathcal{A}} x_{vit} = \text{use}_v, \quad \forall v \in \mathcal{V}, \quad (3)$$

$$\sum_{i: (i,n) \in \mathcal{A}} x_{vin} = \sum_{j: (n,j) \in \mathcal{A}} x_{vnj} = \text{visit}_{vn}, \quad \forall v \in \mathcal{V}, n \in \mathcal{N} \setminus \{s, t\}, \quad (4)$$

$$\text{load}_{vj} \leq \text{load}_{vi} - w_j + M_\ell(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (5)$$

$$\text{soc}_{vj} \leq \text{soc}_{vi} + \text{chg}_{vi} - e_{vij} + M_b(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (6)$$

$$\text{arr}_{vj} \geq \text{arr}_{vi} + \text{srv}_i + \text{chgT}_{vi} + \tau_{ij} - M_t(1 - x_{vij}), \quad \forall v \in \mathcal{V}, (i, j) \in \mathcal{A}, \quad (7)$$

$$\text{arr}_{vn} \geq r_n, \quad \text{tardy}_{vn} \geq \text{arr}_{vn} - d_n, \quad \forall v \in \mathcal{V}, n \in \mathcal{T}. \quad (8)$$

Solve the above MILP using Gurobi/CPLEX, decode the semantic route, and replay the resulting plan in the simulation engine for comparison.

2.4. Frontend and Backend

The frontend layer and the backend layer do not directly participate in path planning or scheduling optimization itself. Instead, they are responsible for organizing the states, events, and statistical results inside the simulation engine into external representations that are accessible, replayable, and analyzable. In this way, the whole system is equipped with a complete

experimental loop and interactive capability.

In implementation, the backend is responsible for unified state packaging and service exposure. The system uses FastAPI and WebSocket to establish a persistent communication channel, and projects the environment state $\mathcal{S}_t$ at time t into frontend-consumable data objects, such as vehicle positions, task states, charging-station load, accumulated score, and event logs. At the same time, the backend also receives control commands

from external users and converts them into invocation requests for the simulation process or algorithm modules. In addition to standard controls such as start, pause, resume, and stop, this layer also provides unified interfaces for Q-learning training and inference, offline MILP solving, and result export, so that the experimental process can be scheduled and reused within the same service framework. In other words, this layer essentially provides a mapping

$$\Psi : \mathcal{S}_t \mapsto \mathcal{O}_t,$$

where $\mathcal{O}_t$ denotes the structured observation result for external interfaces, thereby converting complex low-level states into standardized service outputs.

On this basis, the frontend visualization module further transforms $\mathcal{O}_t$ into graphical expressions that can be easily understood by users. Specifically, the interface continuously displays the spatial distribution of the road network, depots, task nodes, charging stations, and vehicles, while synchronously updating key indicators such as task completion status, vehicle utilization, path cost, charging queue length, and system score. More importantly, this layer does not only serve as a static display component. It also provides a unified interaction entry for policy switching, offline replay, training triggering, and result comparison. As a result, the multi-source information originally scattered across simulation logs, scheduling outputs, and statistical results is reorganized into a visual workflow for analysis and presentation.

3. Experiment

3.1. Experiment Setup

This section introduces the experimental setup from four aspects: the experimental environment, comparison methods, evaluation metrics, and implementation details. Overall, our goal is to systematically evaluate the differences among heuristic baselines, the Q-learning hyper-heuristic, and the offline MILP method in terms of task completion quality, running cost, and scalability within a unified dynamic new energy logistics scheduling environment.

3.1.1. Environment Setup

The environment is built on a graph-based road network and explicitly models a limited number of vehicles, dynamically released tasks, vehicle battery and load constraints, charging-station resources, and task deadlines. Our experiments support three different scales with the following configurations:

- small: includes 5 vehicles, 30 dynamic tasks, 2 charging stations, and 25 road nodes. The map size

is 30×30 , and the simulation horizon is 180. This scale is mainly used for quickly verifying algorithm effectiveness and training convergence behavior.

- medium: includes 10 vehicles, 100 dynamic tasks, 4 charging stations, and 60 road nodes. The map size is 50×50 , and the simulation horizon is 300. This scale is used to test the stability of different methods under higher task density and more complex charging pressure.
- large: includes 20 vehicles, 300 dynamic tasks, 8 charging stations, and 120 road nodes. The map size is 80×80 , and the simulation horizon is 480. In addition, each charging station is equipped with 3 chargers by default. This scale is mainly used to evaluate the scalability and robustness of different methods in complex dynamic scenarios.

In addition to the scale factor, charging decisions can significantly affect vehicle availability, task delay, and the overall score. Therefore, the experiments also consider two charging strategies (optimal_station and nearest_station). The former jointly considers travel distance and station load when selecting a charging station, while the latter always gives priority to the nearest charging station.

3.1.2. Evaluation Metric

To comprehensively evaluate the performance of different methods in the dynamic new energy logistics scenario, we report experimental metrics from two aspects: task effectiveness and operating cost. The definitions of these metrics are given below.

Task effectiveness metrics:

- Completed Tasks: The number of tasks completed by the end of the simulation.
- Expired Tasks: The number of tasks that pass their deadlines and remain unfinished by the end of the simulation.
- Total Tardiness: This metric is used for offline MILP result analysis. It is defined as the sum of task delays over all tasks. If a task is completed before its deadline, its tardiness is recorded as 0.

Operating cost metrics:

- Total Distance: The sum of the travel distances of all vehicles.
- Steps: the number of discrete time steps from the initial time to the end of the simulation in online simulation.
- Makespan: the maximum completion time after all vehicles finish service and return to the depot in offline MILP.
- Charging Burden: We report two statistics: 1) the number of charging triggers, which measures how often the system enters the charging process; and 2)

the average queue load, which measures congestion caused by competition for charging resources.

Training-stage metrics:

- Total Reward: The sum of immediate rewards over all time steps in a single training episode.
- Eval Score Mean: The mean final score obtained by applying the greedy policy over several evaluation episodes after each training round. This metric is used to measure the stable performance of the learned policy, rather than relying only on a single training trajectory.

3.2. Experiment Results

The focus of this experimental analysis includes the performance differences of fixed heuristic baselines across different problem scales, the advantages of the Q-learning hyper-heuristic over the best baseline, the convergence characteristics of Q-learning during training, and the impact of charging strategy design on overall system performance.

3.2.1. Baseline Comparison

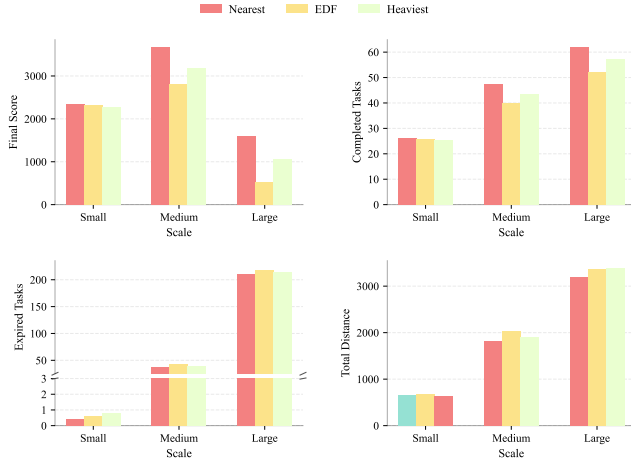


Figure 3. Main comparison of heuristic baselines across three problem scales.

Since the three scenarios contain different numbers of tasks, the absolute number of completed tasks should not be compared across scales directly; instead, the score, expired tasks, and distance should be interpreted together within each scale.

- Small scale: All three heuristics produce close results. Nearest achieves the highest average Final Score of 2354.10, with 26.0 completed tasks and 0.4 expired tasks. EDF follows with a score of 2309.03, and Heaviest obtains 2271.05. This indicates that under light system pressure, all three simple dispatching rules can handle most tasks, and the advantage of any single rule is limited.

- Medium scale: The performance gap becomes clearer. Nearest remains the strongest baseline, achieving an average Final Score of 3662.73, with 47.6 completed tasks and 36.6 expired tasks. Heaviest ranks second with a score of 3187.80, while EDF obtains 2805.43 and produces the most expired tasks among the three baselines. This suggests that deadline priority alone is not sufficient when vehicles also face travel-distance, battery, and charging constraints.
- Large scale: All baselines complete more tasks in absolute number because the scenario contains many more tasks, but the number of expired tasks increases sharply. Nearest still obtains the best score among the three heuristics, with 1605.12 and 61.8 completed tasks, followed by Heaviest with 1054.17 and EDF with 525.16. These results show that fixed rules become increasingly limited under high task density and resource competition, while distance-aware dispatching remains the most robust fixed baseline in the current environment.

3.2.2. Comparison with Q-learning

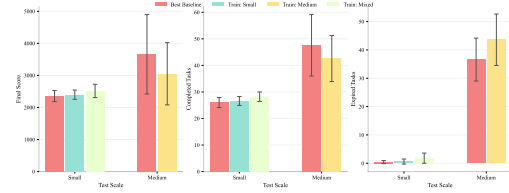


Figure 4. Overall comparison between Q-learning and the best baseline

We compares the learned Q-learning hyper-heuristic with the strongest fixed baseline under the corresponding evaluation settings.

- Small scale: Q-HH-Small achieves a Final Score of 2399.09, with 26.6 completed tasks and 0.6 expired tasks, slightly outperforming the best fixed baseline in terms of score while maintaining a similar task completion level. Q-HH-Mixed also performs well on the small-scale evaluation, obtaining a higher Final Score of 2515.96 and 28.2 completed tasks, but with 1.8 expired tasks on average. This suggests that mixed-scale training can improve task completion and overall score, although it may introduce a small increase in expiration risk.
- Medium scale: Q-HH-Medium achieves an average Final Score of 3050.90, with 42.6 completed tasks and 43.6 expired tasks. Compared with the strongest medium-scale baseline, Nearest, the learned policy is less competitive in the final evaluation runs. This indicates that the medium setting

remains challenging for tabular Q-learning due to the larger state space, denser tasks, and stronger coupling between dispatching and charging.

- Training evidence: The training logs show that Q-HH-Medium reaches a best evaluation score of 4651.62 during training. This demonstrates that Q-learning can discover high-quality rule-selection policies, although their stability across evaluation seeds still needs improvement.

Overall, Figure 4 indicates that Q-learning can be competitive in small-scale and mixed-scale settings, while medium-scale generalization remains more difficult and requires more stable learning or richer state representations.

3.2.3. Training Convergence



Figure 5. Convergence curves of Q-learning under different training scales

We found that Q-learning can improve policy quality during training, but the stability of learning depends strongly on the scenario scale.

- Small scale: The small model has the most stable learning behavior. It reaches a best evaluation score of 2550.60 at episode 155, and the smoothed curves fluctuate less than those of the larger-scale settings. This confirms that tabular Q-learning is suitable when the state space and task interactions are relatively simple.
- Medium scale: The medium model exhibits stronger fluctuations. Although it reaches a high best evaluation score of 4651.62 at episode 230, its final evaluation score decreases. This shows that the learned policy can find high-reward behaviors, but it is less stable under the more complex medium-scale environment.
- Mixed scale: The Mixed model shows a similar pattern. It reaches a best evaluation score of 4455.95 at episode 237, indicating that cross-scale training

can discover strong policies. However, the final smoothed performance still fluctuates because the model alternates between different scenario distributions.

These results suggest that the Q-learning hyper-heuristic is a feasible rule-selection mechanism, but larger or mixed scenarios require more robust training strategies, richer state encoding, or function approximation to improve evaluation stability.

3.2.4. Ablation

Table 1. Charging strategy ablation results on the medium-scale setting.

Method / strategy	Final Score	Completed	Expired	Avg. Queue
Fixed baseline + optimal station	–	47.6	36.6	0.018
Fixed baseline + nearest station	–	47.6	36.6	0.018
Q-learning + optimal station	2935.14	34.6	11.8	–
Q-learning + nearest station	2836.04	33.6	12.0	–

Overall, the charging strategy ablation shows different effects for fixed heuristics and Q-learning.

- Fixed baseline: For the baseline nearest-task scheduler, the two charging strategies obtain almost identical results in the current medium-scale runs. Both strategies complete 47.6 tasks on average, produce 36.6 expired tasks, and have a very low average queue load of about 0.018. This indicates that under the current baseline setting, charging congestion is not the dominant factor limiting performance; task assignment and deadline pressure play a larger role.
- Q-learning with optimal station: Q-learning with optimal station selection achieves an average Final Score of 2935.14, with 34.6 completed tasks and 11.8 expired tasks. This shows that a more informed charging decision can slightly improve the learned policy by reducing expiration and increasing task completion.
- Q-learning with nearest station: Q-learning with nearest station selection obtains a lower Final Score of 2836.04, with 33.6 completed tasks and 12.0 expired tasks. Compared with optimal station selection, the nearest-station rule is simpler but less effective for the learned scheduling policy.

These results suggest that charging strategy design should be considered jointly with task scheduling, but its impact depends on the policy type and congestion level. In the fixed baseline runs, the two station selection strategies behave similarly because charging queues remain short. In the Q-learning runs, optimal station selection provides a clearer benefit, showing that learned scheduling and resource allocation can reinforce each other. Therefore, charging optimization is not merely an auxiliary detail; it can affect the final policy performance, especially when the scheduling

method is adaptive and interacts more frequently with battery and station states.

4. Conclusion

This paper presents a dynamic collaborative scheduling system for new energy logistics fleets based on graph algorithms and reinforcement learning. The system models the urban road network as a weighted graph and builds a discrete-time simulation framework that jointly considers task deadlines, vehicle capacity, battery constraints, and charging-station competition. Based on this framework, we implement and compare classical path-planning algorithms, fixed heuristic scheduling rules, and a Q-learning-based hyper-heuristic that dynamically selects dispatching rules according to the system state. Experimental results across different scenario scales show that fixed heuristics are effective in small-scale settings but become increasingly limited as task density and resource competition grow, while the Q-learning hyper-heuristic can learn useful rule-selection patterns and provide competitive performance in dynamic environments. The results also indicate that charging strategy design should be considered together with task scheduling, because battery recovery and charging queues directly affect vehicle availability and final scheduling quality. Overall, this work integrates graph algorithms, simulation modeling, heuristic scheduling, reinforcement learning, and exact optimization into a unified experimental platform, providing a practical basis for further research on new energy logistics fleet scheduling and multi-agent coordination.

5. Project Reflection

Through this course project, we gained a deeper understanding of how data structures and algorithms can be applied to a realistic and complex logistics scheduling problem. The project required us to connect graph representation, shortest-path search, task scheduling, simulation design, reinforcement learning, experimental evaluation, and paper writing into a complete system, which was much more challenging than implementing isolated algorithms. During development, we learned that algorithmic performance is not determined by a single module alone; path planning, dispatching rules, battery constraints, charging decisions, and evaluation metrics all interact with each other. We also realized the importance of careful experiment design, reproducible data analysis, and consistent explanation between figures, tables, and text. Although the final system still has room for improvement, such as more stable learning methods and richer state representa-

tions, this project helped us improve our ability to design, implement, evaluate, and present a complete algorithmic system, and it gave us a concrete experience of applying data-structure knowledge to real-world intelligent transportation and logistics scenarios.

Dynamic Collaborative Scheduling System for New Energy Logistics Fleets Based on Graph Algorithms and Reinforcement Learning

Supplementary Material

6. Evaluation Metrics

This supplementary section provides the mathematical definitions of the evaluation metrics used in the main paper.

6.1. Task Effectiveness Metrics

Final Score. For each completed task τ_i , its reward is defined as

$$\text{Score}(\tau_i) = \max \left(0, R_0 - \alpha \cdot \min \left(\frac{d_i}{D_{\text{ref}}}, 1 \right) - P_{\text{late}} \cdot \mathbb{I}(t_i^{\text{finish}} > d_i^{\text{deadline}}) \right).$$

Here, R_0 is the base reward, d_i is the service travel distance of task i , D_{ref} is the reference distance used for normalization, α is the normalized distance-cost weight, and P_{late} is the penalty for late completion. In our implementation, we use

$$R_0 = 100, \quad \alpha = 30, \quad P_{\text{late}} = 60.$$

This scoring function does not directly penalize task waiting time. Instead, it considers whether the task is completed, how long the service travel distance is after normalization, and whether the task is completed after its deadline.

The total final score is computed as

$$\text{FinalScore} = \sum_{i \in \mathcal{T}_{\text{completed}}} \text{Score}(\tau_i) - P_{\text{fail}} \cdot |\mathcal{T}_{\text{expired}}| + S_{\text{partial}}.$$

Here, $P_{\text{fail}} = 60$ by default, $\mathcal{T}_{\text{completed}}$ is the set of completed tasks, $\mathcal{T}_{\text{expired}}$ is the set of unfinished expired tasks, and S_{partial} denotes optional partial credit for collaborative tasks.

Completed Tasks. Completed Tasks denotes the number of tasks completed by the end of the simulation:

$$\text{CompletedTasks} = |\mathcal{T}_{\text{completed}}|.$$

Expired Tasks. Expired Tasks denotes the number of tasks that pass their deadlines and remain unfinished by the end of the simulation:

$$\text{ExpiredTasks} = |\mathcal{T}_{\text{expired}}|.$$

Total Tardiness. Total Tardiness is mainly used for offline MILP result analysis and is defined as

$$\text{TotalTardiness} = \sum_n \max(0, \text{arr}_n - d_n),$$

where arr_n denotes the arrival time at task n , and d_n is its deadline.

6.2. Operating Cost Metrics

Total Distance. Total Distance is the sum of travel distances of all vehicles:

$$\text{TotalDistance} = \sum_{v \in \mathcal{V}} \text{dist}_v.$$

Steps. Steps denotes the number of discrete simulation time steps from the initial time to the end of the online simulation.

Makespan. Makespan is used for offline MILP result analysis and denotes the maximum completion time after all vehicles finish service and return to the depot:

$$\text{Makespan} = \max_{v \in \mathcal{V}} T_v^{\text{finish}}.$$

Charging Burden. Charging Burden is measured using two statistics: the number of charging triggers and the average queue load. The number of charging triggers measures how often vehicles enter the charging process, while the average queue load measures congestion caused by competition for charging resources.

6.3. Training-stage Metrics

Total Reward. Total Reward is the sum of immediate rewards over all decision steps in a single Q-learning training episode:

$$\text{TotalReward} = \sum_t r_t.$$

Eval Score Mean. Eval Score Mean is the mean final score obtained by applying the greedy policy over several evaluation episodes after each training round:

$$\text{EvalScoreMean} = \frac{1}{K} \sum_{k=1}^K \text{FinalScore}^{(k)}.$$

This metric measures the stable performance of the learned policy and avoids relying only on a single training trajectory.